

4.13.14

Anshu Kumar Ram - EE25BTECH11009

September 6, 2025

Question

The perpendicular bisector of the line segment joining $P(1, 4)$ and $Q(k, 3)$ has Y -intercept -4 . Then a possible value of k is

- (a) 1 (b) 2 (c) -2 (d) -4 (1)

Theoretical Solution

Equation of perpendicular bisector:

$$\mathbf{n}^T \mathbf{x} = c \quad (2)$$

Direction vector of PQ :

$$\mathbf{PQ} = \begin{pmatrix} k-1 \\ -1 \end{pmatrix} \quad (3)$$

As the perpendicular bisector has normal:

$$\mathbf{n} = \begin{pmatrix} k-1 \\ -1 \end{pmatrix} \quad (4)$$

Midpoint and Substitution

Midpoint of PQ :

$$\mathbf{M} = \begin{pmatrix} \frac{1+k}{2} \\ \frac{7}{2} \end{pmatrix} \quad (5)$$

Y-intercept of the perpendicular bisector:

$$c = -4 \quad (6)$$

Substitute (3) and (6) in (2):

$$\begin{pmatrix} k-1 & -1 \end{pmatrix} \begin{pmatrix} -\frac{1+k}{2} \\ -4 - \frac{7}{2} \end{pmatrix} = 0 \quad (7)$$

Solving for k

$$-(k^2 - 1) + 15 = 0 \quad (8)$$

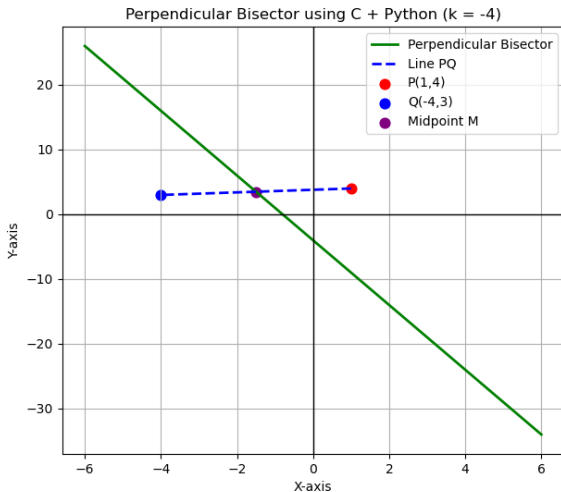
$$-k^2 + 16 = 0 \Rightarrow k^2 = 16 \quad (9)$$

Hence

$$k = \pm 4 \quad (10)$$

From the options, valid value is

$$\boxed{-4}$$



C Code (Part 1)

```
#include <stdio.h>

void compute_bisector(double *P, double *Q, double *results)
{
    double Mx = (P[0] + Q[0]) / 2.0;
    double My = (P[1] + Q[1]) / 2.0;
    double a = Q[0] - P[0]; // x
    double b = Q[1] - P[1]; // y
}
```

C Code (Part 2)

```
double c = a * Mx + b * My;
results[0] = a; // a
results[1] = b; // b
results[2] = c; // c

if (b != 0) {
    results[3] = -a / b; // slope
    results[4] = c / b; // intercept
} else {
    results[3] = 1e9;
    results[4] = 0;
}
```


Python + C Code (Part 1)

```
import numpy as np
import matplotlib.pyplot as plt
import ctypes

lib = ctypes.CDLL("./func.so")
lib.compute_bisector.argtypes = [
    ctypes.POINTER(ctypes.c_double),
    ctypes.POINTER(ctypes.c_double),
    ctypes.POINTER(ctypes.c_double)
]
```

Python + C Code (Part 2)

```
P = np.array([1.0, 4.0])
Q = np.array([-4.0, 3.0])
results = (ctypes.c_double * 5)()
P_c = (ctypes.c_double * 2)(*P)
Q_c = (ctypes.c_double * 2)(*Q)

lib.compute_bisector(P_c, Q_c, results)
a, b, c, slope, y_intercept = [results[i] for i in range(5)]

M = (P + Q) / 2
x_vals = np.linspace(-6, 6, 100)
if abs(slope) < 1e8:
    y_vals = slope * x_vals + y_intercept
else:
    x_vals = np.ones(100) * (-c / a)
    y_vals = np.linspace(-6, 6, 100)
```

Python + C Code (Part 3)

```
plt.figure(figsize=(7, 6))
plt.plot(x_vals, y_vals, 'g-', lw=2, label='Perpendicular
        Bisector')
plt.plot([P[0], Q[0]], [P[1], Q[1]], 'b--', lw=2, label='Line PQ'
        )

plt.scatter(*P, color='red', s=60, label='P(1,4)')
plt.scatter(*Q, color='blue', s=60, label='Q(-4,3)')
plt.scatter(*M, color='purple', s=60, label='Midpoint M')

plt.axhline(0, color='black', lw=1)
plt.axvline(0, color='black', lw=1)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Perpendicular Bisector using C + Python (k = -4)")
plt.legend()
plt.grid(True)
plt.show()
```

Pure Python Code (Part 1)

```
import numpy as np
import matplotlib.pyplot as plt

# Given points
P = np.array([1, 4])
k = -4 # Correct answer
Q = np.array([k, 3])

# Midpoint of PQ
M = (P + Q) / 2

# Direction vector of PQ
PQ = Q - P

# Normal vector to bisector (same as direction of PQ)
n = PQ

# Equation of bisector:  $n(x - M) = 0$     $a*x + b*y = c$ 
a, b = n
c = a*M[0] + b*M[1]
```

Pure Python Code (Part 2)

```
# Find slope and y-intercept
if b != 0:
    m_bisector = -a/b
    y_intercept = c/b
else:
    m_bisector = None # vertical line case

print(f"Equation of perpendicular bisector:")
print(f"({a})x + ({b})y = {c:.2f}")
if m_bisector is not None:
    print(f"y = {m_bisector:.2f}x + {y_intercept:.2f}")

# Generate x values for plotting
x_vals = np.linspace(-6, 6, 100)
y_vals = m_bisector * x_vals + y_intercept
```

Pure Python Code (Part 3)

```
plt.figure(figsize=(7, 6))
plt.plot(x_vals, y_vals, 'g-', lw=2, label='Perpendicular
        Bisector')
plt.plot([P[0], Q[0]], [P[1], Q[1]], 'b--', lw=2, label='Line PQ'
        )

# Mark points
plt.scatter(*P, color='red', s=60, label='P(1,4)')
plt.scatter(*Q, color='blue', s=60, label='Q(-4,3)')
plt.scatter(*M, color='purple', s=60, label='Midpoint M')

plt.axhline(0, color='black', lw=1)
plt.axvline(0, color='black', lw=1)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Perpendicular Bisector (Pure Python, k = -4)")
plt.legend()
plt.grid(True)
plt.show()
```